

LAB REPORT 12

Pointers in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

This lab was designed to build a working understanding of pointers — one of the most distinctive features of C++. The session aimed to achieve the following:

- Grasping the relationship between variables, memory bytes, and addresses
- Declaring pointer variables and understanding what they actually store
- Using the **address-of** operator (&) to obtain a variable's location in memory
- Using the **dereference** operator (*) to read and write the value at a given address

2 Theoretical Background

Memory, Bytes, and Addresses

Every piece of data a program works with has to live somewhere in RAM. RAM is organised as a long sequence of individual bytes, and each byte has its own unique numeric identifier called an address. When the compiler creates a variable, it reserves a block of consecutive bytes and gives them to that variable. An `int`, for instance, takes up four of those bytes on most platforms. Knowing this layout is the foundation on which pointer logic is built.

Pointers — Variables That Hold Addresses

A **pointer** is a variable whose job is to store an address rather than an ordinary value. While an `int` holds a number like 42, an `int*` holds the address of the memory cell where 42 is sitting. This indirection gives the programmer a way to reach any location in memory directly, which is essential for tasks like building dynamic data structures and managing heap-allocated memory.

Getting an Address with &

The `&` symbol, when placed before a variable name in an expression, yields the address of that variable rather than its value. Executing `ptr = &k;` reads the address where `k` resides and writes that address into `ptr`. From that point on, `ptr` knows exactly where `k` is stored in memory.

Reading and Writing Through a Pointer with *

Putting `*` in front of a pointer in an expression tells the program to follow the stored address and operate on whatever is there. `cout << *ptr` prints the value found at that address, while `*ptr = 7` writes the number 7 into that same memory slot — directly modifying the original variable without using its name at

all.

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — Pointer Declaration and Initialisation

An integer was set up and given a starting value. A pointer of the same type was then created and linked to that integer by storing its address. Three values were printed — the number read directly, the address stored in the pointer, and the number reached through the pointer — to verify all three routes give consistent results.

● ● ● C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int num = 42;
    int* ptr = &num;      // pointer stores address of num

    cout << "Value of num:      " << num << endl;
    cout << "Address of num:    " << ptr << endl;
    cout << "Value via pointer:  " << *ptr << endl;
    return 0;
}
```

Task 2 — Modifying a Variable Through a Pointer

Rather than using a variable's name to update it, this task used a pointer to reach into the variable's memory slot and overwrite its content. The value was printed before and after the pointer-based assignment to make the effect visible.

● ● ● C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int k = 10;
    int* ptr = &k;

    cout << "Before: k = " << k << endl;
    *ptr = 99;           // modify k through the pointer
    cout << "After:  k = " << k << endl;
    return 0;
}
```

Task 3 — Pointer Arithmetic with Arrays

A five-element integer array was filled with values and a pointer was aimed at its first element. Inside a loop, the pointer was advanced mathematically — adding the loop index to its address — to reach each subsequent element without ever using the subscript operator.

● ● ● C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* ptr = arr;          // points to first element

    for (int i = 0; i < 5; i++) {
        cout << "arr[" << i << "] = " << *(ptr + i) << endl;
    }
    return 0;
}
```

5 Conclusion

This lab brought the abstract idea of memory addresses into sharp focus. Rather than letting the compiler handle all memory details invisibly, pointers expose those details and give the programmer direct control. The three tasks showed how to set up a pointer, how to reach through it to alter a variable's contents, and how array traversal looks at the address level. These skills underpin almost everything that comes later in systems programming — from heap allocation to linked structures.